MINI PROJECT

COMPUTATIONAL INTELLIGENCE

MCTA 3371

DR. AZHAR BIN MOHD IBRAHIM

DR. AMIR AKRAMIN BIN SHAFIE

SECTION 2

MATRIC NUMBER	NAME
2312479	MUHAMMAD HAZIMUDDIN BIN FAIRUZ
2312451	MUHAMMAD ZAMARUL AZIM BIN ZULKHIBRI
2316963	MUHAMMAD AIMAN ALFARUQ BIN NOREDAN

DATE OF SUBMISSION

Saturday, 24th January 2026

Abstract

Mobile robot navigation requires intelligent decision-making to safely move from a start location to a goal while avoiding obstacles in complex environments. This project presents the development of an intelligent navigation system using a hybrid Computational Intelligence approach that combines Fuzzy Logic and Genetic Algorithm (GA) optimization. The robot is modelled as a point robot operating in a two-dimensional environment with discrete step-based movement and five virtual sensors that detect obstacle distances and goal direction.

A baseline fuzzy logic controller is first designed using triangular membership functions, linguistic rule bases, and centroid defuzzification to determine steering and movement decisions. To enhance the controller's performance and remove dependency on manual tuning, a Genetic Algorithm is integrated to automatically optimize key fuzzy membership parameters. This hybrid Fuzzy–GA controller adapts the fuzzy system to produce more efficient navigation behaviour.

Two different environments, namely a Simple Map and a Complex Map, are created to evaluate the robustness of the controllers under varying obstacle densities and narrow passages. The performance of the baseline fuzzy controller and the GA-optimized fuzzy controller is compared using evaluation metrics including goal-reaching success rate, path efficiency, number of collisions, and time steps required to reach the goal.

Simulation results demonstrate that the hybrid Fuzzy–GA controller significantly improves navigation performance, especially in complex environments, by producing smoother paths, fewer collisions, and faster goal reaching compared to the manually tuned fuzzy controller. The project highlights how hybrid computational intelligence techniques can enhance autonomous robot navigation without relying on complex mathematical models.

Keywords — Mobile robot navigation, Fuzzy logic, Genetic algorithm, Hybrid computational intelligence, Obstacle avoidance, Discrete navigation, Path efficiency.

Table of Contents

Abstract..... 2
Table of Contents.....3
1.0 Introduction..... 4
1.1 Real World Application of Mobile Robot Navigation..... 4
1.2 Suitability of Fuzzy Logic for Robot Navigation.....4
1.3 Motivation for Hybrid Fuzzy–Genetic Algorithm (GA).....4
1.4 Project Overview.....5
2.0 System Architecture and Environment Setup..... 5
2.1 Software Packages and Requirements.....5
2.2 Robot Model.....5
2.2.1 Movement Model..... 6
2.2.2 Sensors..... 6
2.3 Environment and Obstacle Maps..... 6
2.3.1 Simple Map..... 6
2.3.2 Complex Map.....7
3.0 Soft Computing Modeling.....8
3.1 Base Fuzzy Logic Controller.....8
3.1.1 Inputs.....8
3.1.2 Outputs..... 8
3.1.3 Membership Functions.....8
3.1.4 Rule Base.....8
3.1.5 Decision-Making Process.....9
3.2 Hybrid Fuzzy–Genetic Algorithm Controller..... 9
3.2.1 Rationale for Using Genetic Algorithm..... 9
3.2.1 Rationale for Using Genetic Algorithm..... 9
3.2.2 Optimization Parameters..... 9
3.2.3 Fitness Function..... 9
3.2.4 Hybridization Benefits..... 10
4.0 Simulation and Implementation..... 10
5.0 Results and Discussion.....14
6.0 Conclusion and Future Works..... 16
7.0 Reference.....17

1.0 Introduction

Mobile robot navigation refers to the capability of a robot to autonomously move from a starting location to a designated target while avoiding obstacles in its surrounding environment. This capability is a fundamental requirement in modern robotics, particularly for applications that operate in dynamic, uncertain, or cluttered environments. Effective navigation requires intelligent decision-making mechanisms that allow robots to respond adaptively to environmental changes while ensuring safety and efficiency.

1.1 Real World Application of Mobile Robot Navigation

Mobile robot navigation has been widely applied in various real-world systems. Indoor service robots, such as cleaning robots, delivery robots, and hospital assistant robots, rely on autonomous navigation to operate efficiently in human-centered environments. In industrial settings, Automated Guided Vehicles (AGVs) are extensively used in warehouses and factories for material handling and logistics automation. Additionally, mobile robots are deployed in search and rescue missions to operate in hazardous or inaccessible environments, as well as in autonomous vehicle systems navigating structured and semi-structured environments.

These applications often involve uncertainty, incomplete sensor information, and continuously changing surroundings, making the use of traditional model-based control approaches challenging and less effective.

1.2 Suitability of Fuzzy Logic for Robot Navigation

Fuzzy Logic is particularly well-suited for mobile robot navigation as it enables human-like reasoning through linguistic rules rather than relying on precise mathematical models. It effectively handles uncertainty, imprecision, and noisy sensor data, which are common in real-world navigation scenarios. Furthermore, fuzzy logic controllers are relatively simple to design and do not require accurate kinematic or dynamic models of the robot, making them attractive for real-time navigation applications.

1.3 Motivation for Hybrid Fuzzy–Genetic Algorithm (GA)

Despite its advantages, the performance of a fuzzy logic controller is highly dependent on the design of its membership functions and rule parameters, which are often determined through manual tuning and expert intuition. This manual process can be

time-consuming and may not yield optimal performance across different environments. Genetic Algorithms (GA), inspired by the principles of natural evolution, provide an effective optimization framework for automatically tuning fuzzy parameters. By integrating fuzzy logic with GA, the hybrid controller aims to enhance navigation performance, improve adaptability to different environments, and reduce reliance on manual parameter tuning.

1.4 Project Overview

This project focuses on the design and simulation of an intelligent mobile robot navigation system using a hybrid Fuzzy Logic and Genetic Algorithm approach. A point robot is simulated to navigate in two-dimensional environments containing obstacles, with the objective of reaching a predefined goal while avoiding collisions. The performance of a baseline fuzzy logic controller is compared against a hybrid fuzzy–GA controller across different simulated maps. The effectiveness of each controller is evaluated based on navigation performance metrics such as goal-reaching success, time steps, and overall navigation efficiency. The simulation-based approach allows systematic evaluation of navigation behaviour without the complexity of physical robot hardware.

2.0 System Architecture and Environment Setup

The simulation environment and navigation controller were developed using Python within the PyCharm Integrated Development Environment (IDE). Python was selected due to its flexibility, extensive library support, and suitability for rapid prototyping of computational intelligence algorithms. PyCharm further facilitated efficient code development, debugging, and visualization during the simulation process.

The overall system architecture follows a perception–decision–action loop. Sensor information from the environment is first perceived by the robot, processed by the fuzzy logic controller to generate motion decisions, and subsequently executed as movement actions. The resulting changes in the robot's position are then evaluated through continuous interaction with the environment, forming a closed-loop navigation system.

2.1 Software Packages and Requirements

The following Python libraries were used in the implementation:

- **NumPy**: Numerical computations and vector operations
- **Matplotlib**: Visualization of maps, robot trajectories, and performance results
- **Pygame**: Real-time 2D simulation and animation of the robot and environment
- **Scikit-fuzzy**: Implementation of fuzzy logic systems

These packages provide sufficient functionality to simulate mobile robot navigation and to evaluate the performance of different navigation controllers.

2.2 Robot Model

The mobile robot is modelled as a point robot, which simplifies the navigation problem by neglecting physical dimensions while preserving essential motion and decision-making characteristics. This abstraction allows the study to focus on the intelligence of the navigation controller rather than detailed robot kinematics or dynamic modelling. Despite its simplicity, the point robot model is sufficient to evaluate navigation strategies and obstacle avoidance behaviour in two-dimensional environments.

2.2.1 Movement Model

The robot operates using a discrete movement model. At each simulation step, the robot executes one of several predefined actions, namely moving forward, turning left, or turning right. Each action results in an update of the robot's position and orientation within the environment. This step-based motion model reflects a deliberative navigation strategy, where decisions are made incrementally based on the robot's current sensory information and controller output.

2.2.2 Sensors

To perceive its surroundings, the robot is equipped with simplified virtual sensors. These sensors provide information on the distance to the goal, the relative angle between the robot's current heading and the direction of the goal, and the distances to the nearest obstacles in the front, left, and right directions. The sensor readings serve as inputs to the fuzzy logic

controller, enabling the robot to make informed movement decisions while avoiding collisions and progressing toward the target, as illustrated in **Figure 2.2.2**.

The robot does not possess prior knowledge of the global map layout. All navigation decisions are made reactively based solely on local sensor information at each simulation step. To focus on the effectiveness of the navigation controller, the sensors are assumed to be ideal and noise-free in this simulation.

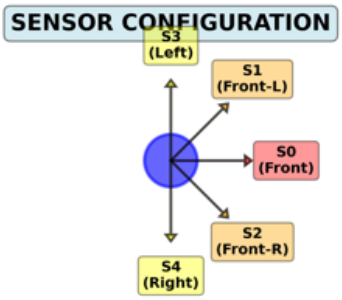

Figure 2.2.2 Shows the sensor configuration

2.3 Environment and Obstacle Maps

Two different environments were designed to evaluate the performance and robustness of the navigation controllers under varying levels of complexity. Each environment contains a predefined start location and a goal location that the robot must reach. The use of multiple environments allows the assessment of controller adaptability under both simple and challenging navigation scenarios.

2.3.1 Simple Map

The Simple Map consists of sparsely placed obstacles and a relatively direct path from the start point to the goal. This environment is primarily used to validate the basic navigation capability of the controllers and to ensure that the robot can successfully reach the target under low-complexity conditions.

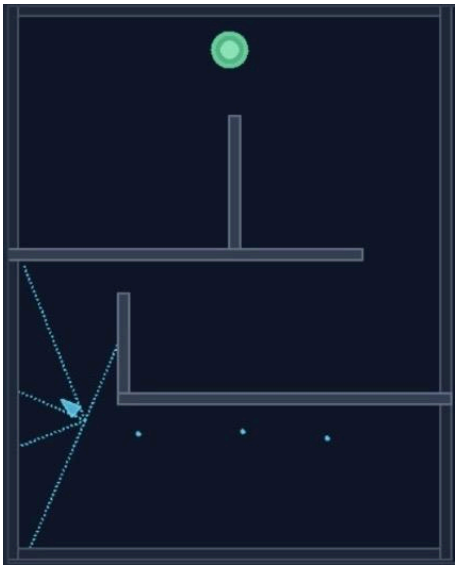

Figure 2.3.1 Shows the Simple Map configuration

2.3.2 Complex Map

In contrast, the Complex Map features dense and irregular obstacle arrangements, including narrow passages and sharp turns. This environment is designed to challenge the controller's adaptability and obstacle avoidance capabilities. It is therefore well suited for evaluating the effectiveness of the hybrid fuzzy–genetic algorithm approach under more demanding navigation conditions.

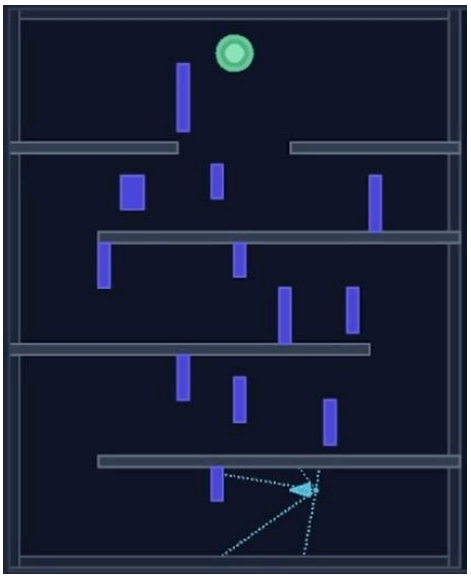

Figure 2.3.2 Shows the Complex Map configuration

3.0 Soft Computing Modelling

3.1 Base Fuzzy Logic Controller

The base navigation controller is implemented using a fuzzy logic system that determines the robot's movement based on real-time sensory inputs. Fuzzy logic is selected due to its ability to handle uncertainty and imprecision in sensor information while enabling human-like reasoning without requiring explicit mathematical models of the robot's kinematics or dynamics.

The controller operates within a discrete, step-based navigation framework. At each simulation step, the robot evaluates its current sensory inputs and generates motion commands through fuzzy inference, which are then executed as incremental movements in the environment.

3.1.1 Inputs

The fuzzy navigation controller is designed using two primary inputs: obstacle distance information and the goal bearing angle. These inputs provide sufficient environmental awareness and directional guidance for intelligent navigation and obstacle avoidance.

3.1.1.1 Distance Sensor Input

The first input represents the robot's perception of obstacle proximity in its immediate surroundings. This value is derived from virtual distance sensors that measure the distance to the nearest obstacles in the robot's sensing directions.

The distance input is fuzzified into three linguistic variables using triangular membership functions, as illustrated in **Figure 3.1.1.1**:

- Near
- Medium
- Far

In the baseline fuzzy controller, representative membership ranges are defined to reflect different levels of obstacle proximity. Smaller distance values activate the *Near* region,

indicating a high collision risk, while larger distance values activate the *Far* region, indicating a clear path ahead. These membership parameters are later optimized in the hybrid fuzzy–genetic algorithm approach described in Section 3.2. This fuzzification scheme allows the robot to smoothly transition between obstacle avoidance behaviour and forward motion as environmental conditions change.

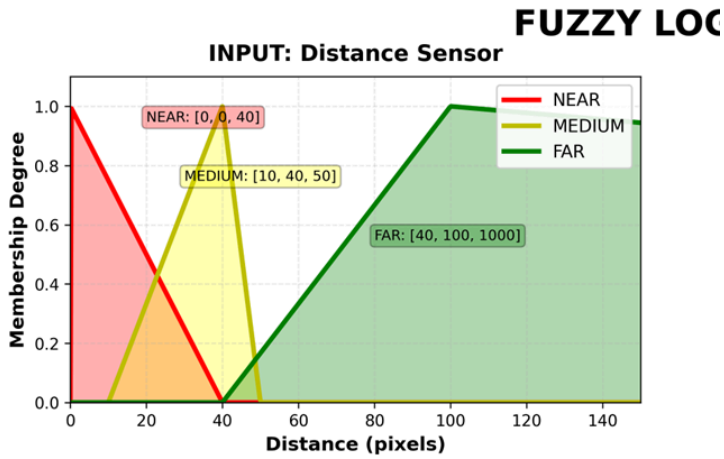

Figure 3.1.1.1 First Input(Distance Sensor) Graph

3.1.1.2 Goal Bearing Angle Input

The second input represents the relative angle between the robot's current heading and the direction of the goal. This angle is expressed in radians and fuzzified into three linguistic variables, as shown in **Figure 3.1.1.2**:

- **Clockwise (CW)**
- **Straight**
- **Counter-Clockwise (CCW)**

Negative angle values activate the *CW* membership, angles close to zero activate *Straight*, and positive angle values activate *CCW*. These membership functions enable the robot to reason about the relative direction of the goal and adjust its steering accordingly.

By converting crisp sensor readings into linguistic variables, the fuzzy controller can perform intuitive, rule-based reasoning without relying on precise mathematical formulations.

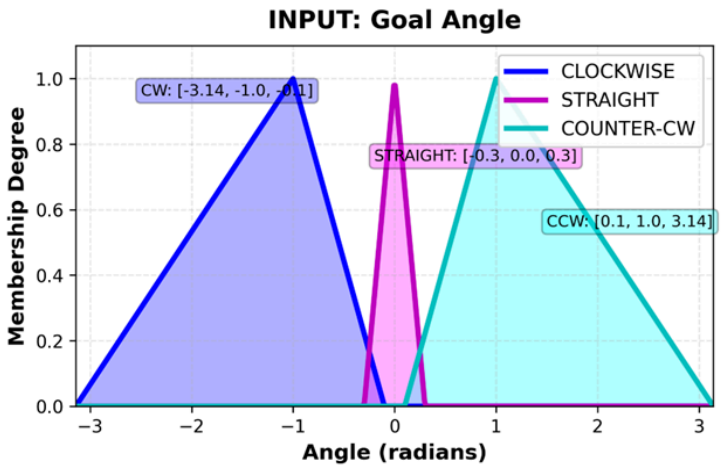

Figure 3.1.1.2 Second Input Graph

3.1.2 Outputs

Based on the evaluated inputs, the fuzzy logic controller generates two continuous output variables: steering direction and movement velocity. The steering output determines the direction of rotation, while the velocity output influences the forward movement magnitude of the robot. These fuzzy outputs are defuzzified into crisp values and subsequently converted into discrete motion commands that update the robot's position and orientation within the environment.

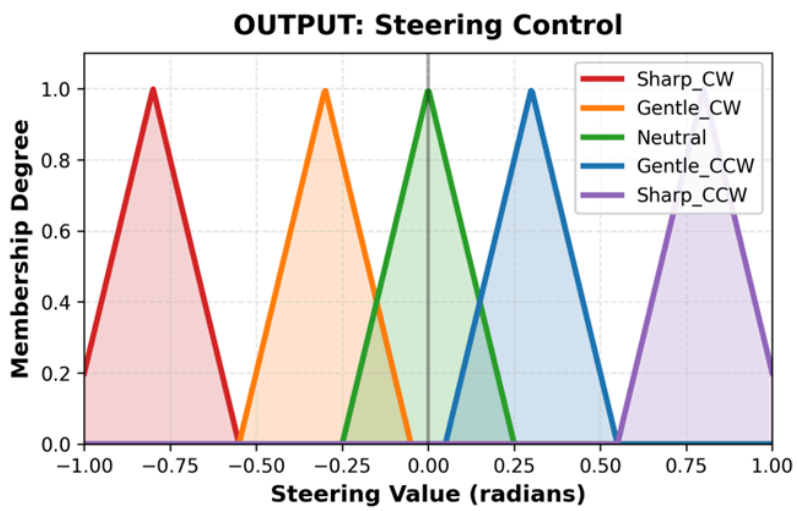

Figure 3.1.2.1 First Output Graph

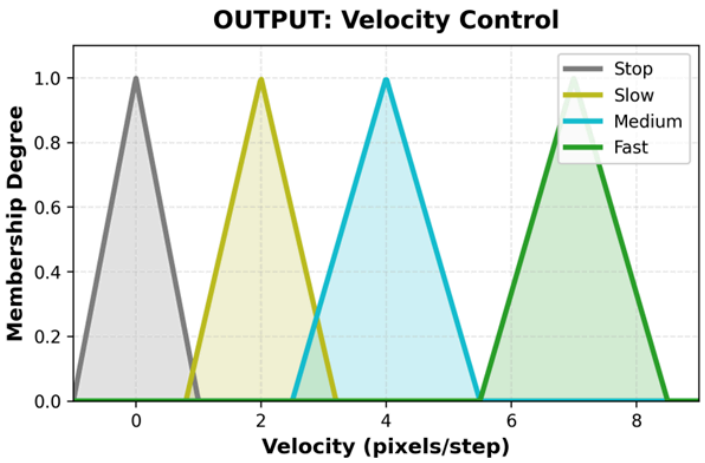

Figure 3.1.2.2 Second Output Graph

3.1.3 Membership Functions

Triangular membership functions are used for both input and output variables due to their simplicity and computational efficiency. These functions map crisp values into degrees of membership, enabling smooth transitions between control actions and preventing abrupt or unstable robot movements.

The membership functions are designed to reflect intuitive navigation behaviour and are visualized through membership plots generated during simulation.

3.1.4 Rule Base

The fuzzy rule base consists of intuitive IF–THEN rules derived from common navigation behaviour. Examples include:

- IF front obstacle is Near THEN turn away from the closer side
- IF path ahead is clear AND goal is to the left THEN gently turn left
- IF obstacles on both sides are Near THEN move straight slowly
- IF path is clear AND goal is straight THEN move forward faster

These rules combine obstacle avoidance and goal-seeking behaviour within a single decision-making framework. The rule base remains fixed throughout the project.

3.1.5 Decision-Making Process

At each simulation step, the navigation decision-making process follows these stages:

1. Sensor readings are fuzzified using triangular membership functions.
2. The fuzzy rule base is evaluated to determine rule activation strengths.
3. Output fuzzy sets are aggregated.
4. Centroid defuzzification is applied to obtain crisp steering and velocity values.
5. The resulting outputs are converted into discrete motion commands, and the robot executes the movement.

This process repeats iteratively until the robot successfully reaches the goal or fails due to a collision.

3.2 Hybrid Fuzzy–Genetic Algorithm Controller

3.2.1 Rationale for Using Genetic Algorithm

Although fuzzy logic provides an effective rule-based navigation strategy, its performance is highly dependent on the appropriate tuning of membership function parameters. Manual tuning of these parameters is often time-consuming and may not yield optimal behaviour, particularly in complex environments containing narrow passages and dense obstacle configurations.

To address this limitation, a Genetic Algorithm (GA) is incorporated to automatically optimize selected fuzzy membership parameters. GA is an evolutionary optimization technique inspired by the principles of natural selection, capable of exploring a wide range of parameter combinations through population-based search and iterative improvement.

In this project, the GA enhances the fuzzy controller by optimizing membership function parameters without altering the fuzzy rule base or output structure. This approach preserves the interpretability and transparency of the fuzzy logic system while improving its navigation performance.

3.2.2 Optimization Parameters

The GA optimizes four key parameters that define the triangular membership functions of the distance sensor input:

- Near distance threshold
- Medium range start value
- Medium range end value
- Far range start value

These parameters directly influence how the robot interprets obstacle proximity in its environment. Variations in these values significantly affect navigation behaviour, especially in confined spaces where precise obstacle avoidance is critical.

Each chromosome in the GA represents a unique combination of these four parameters, forming a candidate fuzzy controller configuration to be evaluated during training.

3.2.3 GA Training Process

The GA training process follows a standard evolutionary optimization framework:

1. An initial population of chromosomes, each representing a candidate set of fuzzy membership parameters, is randomly generated.
2. Each chromosome is used to configure the fuzzy logic controller.
3. The robot navigates the environment using the configured controller.
4. Navigation performance is evaluated using a fitness function.
5. Selection, crossover, and mutation operators are applied to generate new populations with improved parameter sets.

Through successive generations, the GA gradually evolves parameter combinations that result in more effective and reliable navigation behaviour.

3.2.4 Fitness Function

The fitness function evaluates the quality of navigation performance during GA training based on task-level outcomes. It considers factors such as:

- Successful reaching of the goal
- Shorter path length
- Reduced number of collisions

- Fewer time steps required to reach the goal

Controllers that successfully reach the goal with fewer steps and fewer collisions are assigned higher fitness values and are more likely to be selected for reproduction in subsequent generations. This performance-driven evaluation encourages the evolution of safer and more efficient navigation strategies.

3.2.5 Hybridization Benefits

By integrating GA with the fuzzy logic controller, the navigation system gains adaptive capability without sacrificing the transparency of rule-based decision-making. The optimized membership functions improve the robot's ability to interpret obstacle proximity more effectively, leading to observable improvements such as:

- Enhanced obstacle avoidance in narrow and cluttered environments
- Reduced oscillatory or unnecessary turning behaviour
- More consistent navigation trajectories
- Improved success rates and reduced time steps to reach the goal

These results demonstrate that the hybrid fuzzy–GA approach provides superior navigation performance compared to the manually tuned baseline fuzzy controller.

4.0 Simulation and Implementation

The mobile robot navigation system was implemented and simulated entirely in Python using the PyCharm Integrated Development Environment. The implementation emphasizes the evaluation of intelligent navigation behaviour and controller performance rather than graphical realism. Several Python libraries were utilized to support the simulation process. NumPy was used for numerical and vector computations, Pygame was employed to render the two-dimensional simulation environment and visualize robot motion, while scikit-fuzzy was used to implement the fuzzy decision-making process, including fuzzification, rule evaluation, and defuzzification.

The simulation operates in a discrete-time framework. At each simulation step, the robot's sensor readings are updated, the navigation controller generates motion commands, and the robot's position and orientation are updated accordingly. The robot's trajectory is traced throughout the simulation to allow qualitative analysis of navigation behaviour and

path efficiency. Collision detection is performed by checking whether the robot's position intersects with obstacle boundaries. Each simulation run is terminated when one of the following conditions is met:

- The robot successfully reaches the goal location.
- The robot collides with an obstacle.
- A predefined maximum number of steps is exceeded.

This termination strategy ensures consistent and fair evaluation conditions across all simulation runs.

4.1 Genetic Algorithm Training Setup

The Genetic Algorithm is used to optimize selected fuzzy membership function parameters through offline evolutionary optimization. During training, each chromosome represents a candidate fuzzy controller configuration. The robot navigates the environment using the parameters defined by the chromosome, and navigation performance is evaluated to assign a fitness value.

In the Simple Map environment, the GA training process is illustrated in **Figure 4.1**. This environment provides a baseline setting for observing the effect of parameter optimization under low environmental complexity. Following training, the navigation behaviour of the Baseline Fuzzy Controller and the GA-optimized Fuzzy Controller is compared under identical conditions, as shown in **Figure 4.2**.

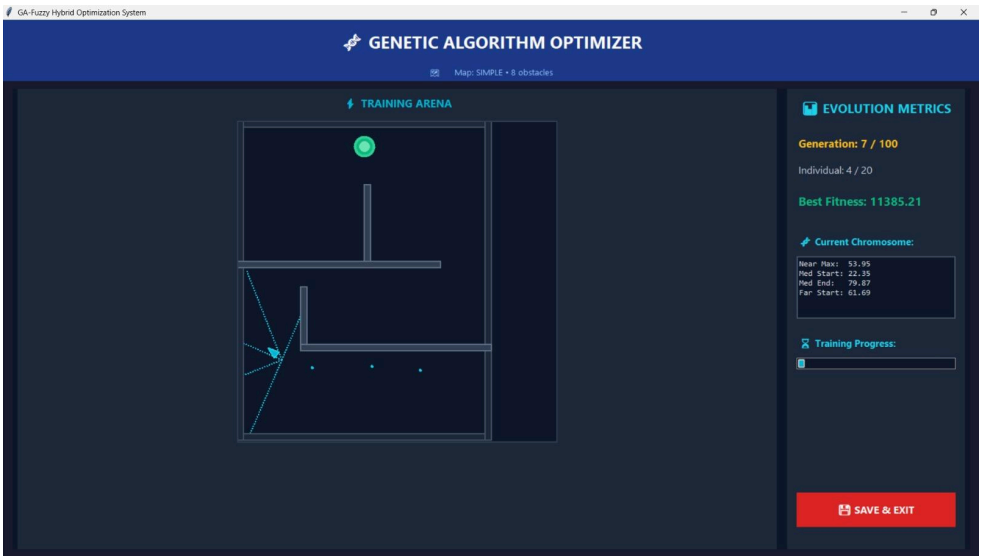

Figure 4.1

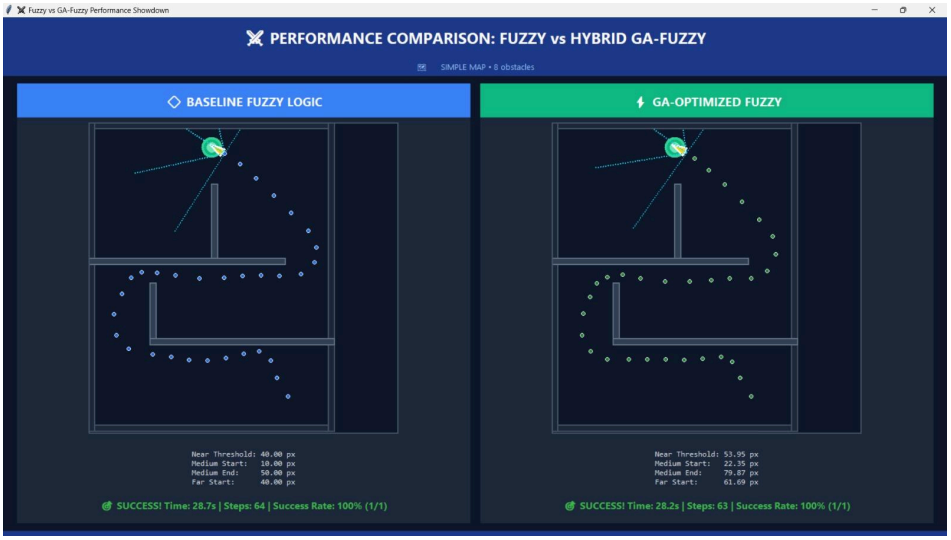

Figure 4.2

The Complex Map presents a more challenging navigation scenario with dense obstacles, narrow corridors, and sharp turns. To investigate the influence of evolutionary optimization depth, the GA was trained using different numbers of generations. When a low number of generations was used, the fuzzy membership parameters were only partially optimized. The GA training process under this condition is shown in **Figure 4.3**, while the corresponding navigation comparison between the Baseline Fuzzy Controller and the GA-optimized controller is presented in **Figure 4.4**.

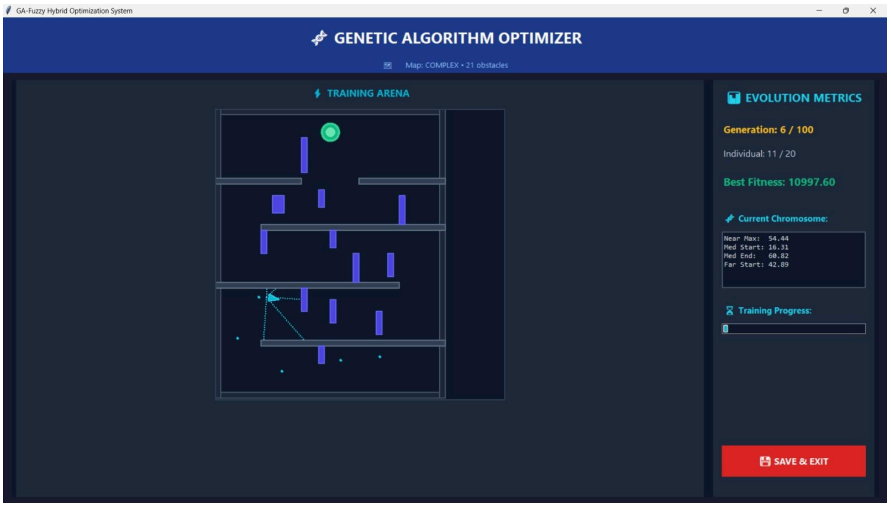

Figure 4.3

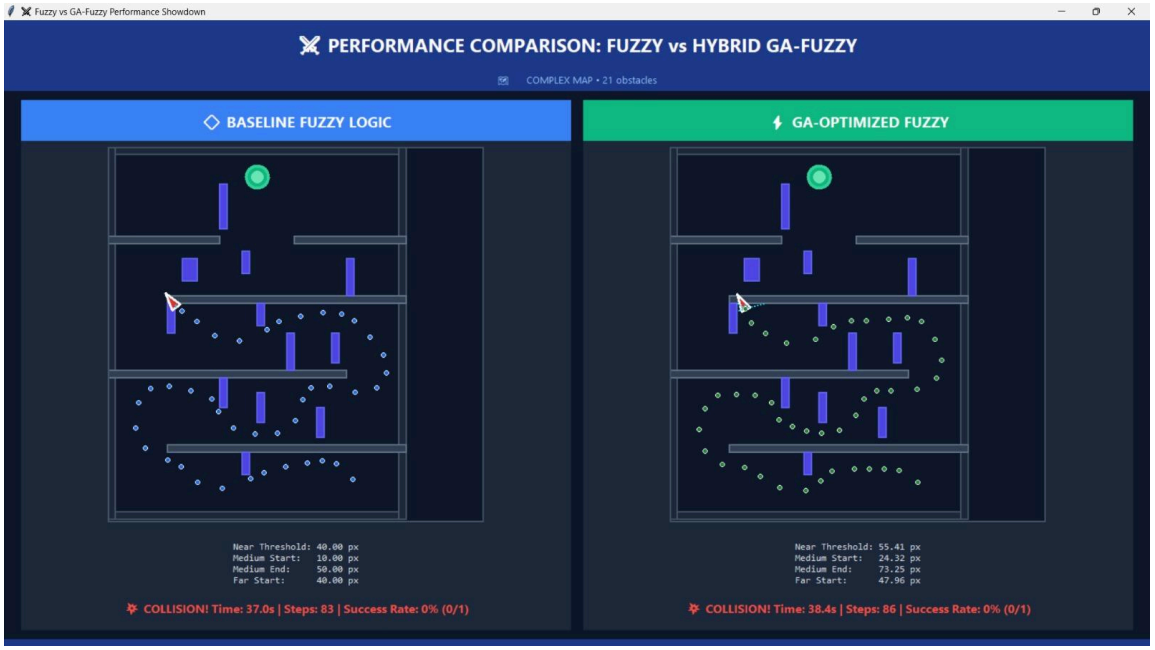

Figure 4.4

To further analyze the impact of extended evolutionary optimization, the GA was trained with a higher number of generations in the Complex Map. This allows the fuzzy membership parameters to converge toward more effective values. The training process for the high-generation case is illustrated in **Figure 4.5**, and the resulting navigation comparison is shown in **Figure 4.6**.

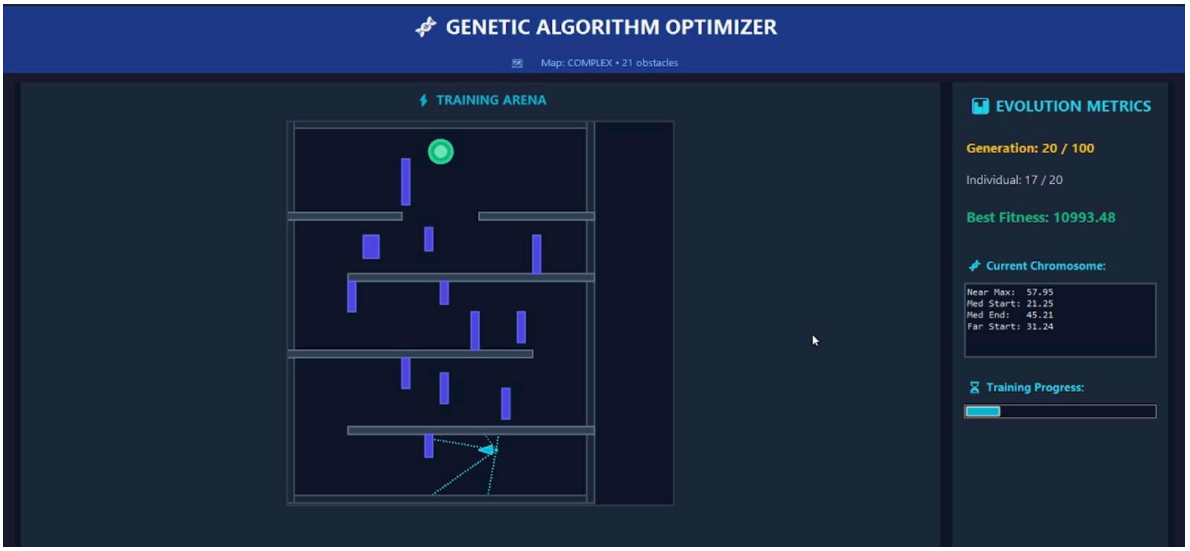

Figure 4.5

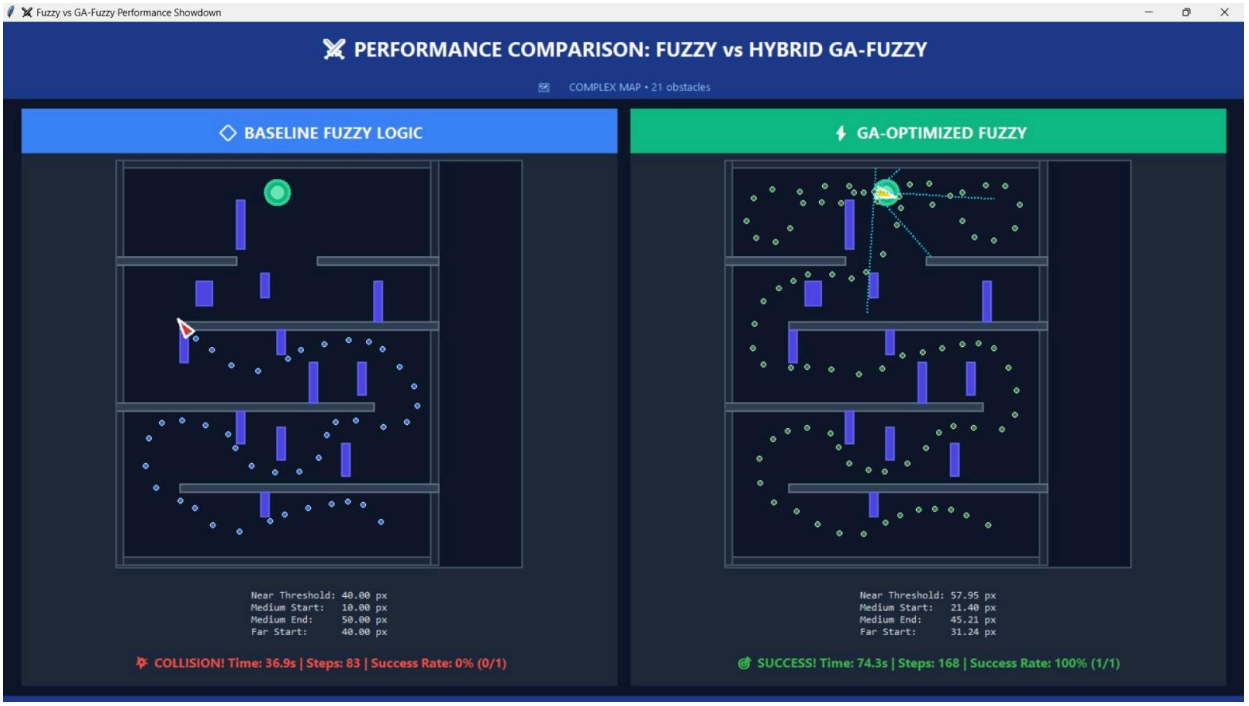

Figure 4.6

Throughout all simulations, identical environment configurations, start and goal locations, and termination criteria were used to ensure fair comparison between the Baseline Fuzzy Controller and the Hybrid Fuzzy–Genetic Algorithm Controller. By maintaining consistent evaluation conditions, performance differences observed in the results can be attributed directly to the effect of fuzzy parameter optimization rather than environmental variation.

5.0 Results and Discussion

The performance of the mobile robot was tested in both Simple and Complex map environments. The results of the Baseline Fuzzy Controller and the Hybrid GA–Fuzzy Controller were compared to understand how each method performs under different conditions.

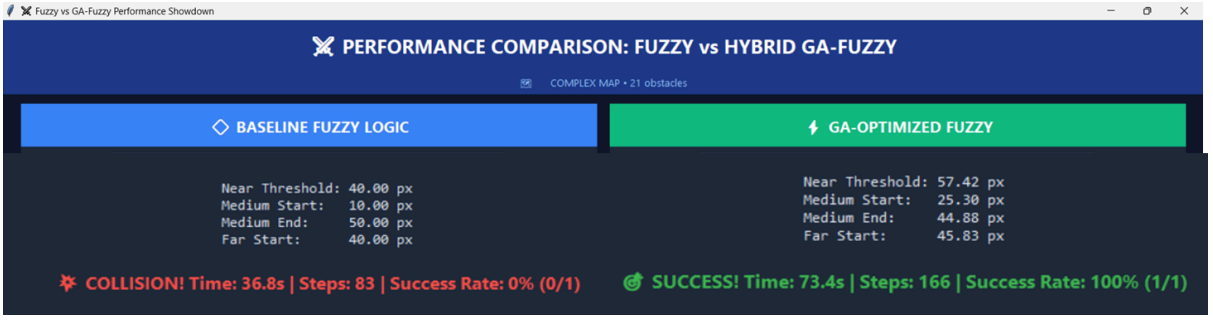

Figure 5.0 Result Before & After Implement Genetic Algorithm

5.1 Results in Simple Map

The results of the Simple Map are shown in Table 1. Both the Baseline Fuzzy controller and the GA–Fuzzy controller were able to reach the goal successfully without any collisions. The paths taken by the robot in both cases were almost the same, and the overall performance was good for both controllers. This shows that in a simple environment with fewer obstacles, a basic fuzzy logic controller is enough to guide the robot safely to the target. Since the environment is not very challenging, the optimization provided by the Genetic Algorithm does not make a major difference. Therefore, both controllers perform equally well in simple maps.

Table 1: Result for Simple Map

Metric	Baseline Fuzzy	GA-Fuzzy
Goal Reached	Consistently	Consistently
Collisions	Non observed	Non observed
Path Length	Similar	Similar
Performance in Simple Map	Good	Good

5.2 Results in Complex Map (Low GA Generation)

The navigation performance in the Complex Map with a low number of GA generations is shown in **Table 2**. In this environment, the Baseline Fuzzy Controller performed poorly. The robot frequently collided with obstacles and was unable to reach the goal in most trials. This behaviour highlights the limitation of fixed fuzzy parameters when dealing with narrow passages and dense obstacle configurations.

When the GA–Fuzzy Controller was applied with a limited number of generations, a modest improvement was observed. The robot was able to avoid some obstacles more effectively and reached the goal in a small number of trials. However, collisions were still frequent, and navigation performance remained inconsistent. These results suggest that a low number of GA generations is insufficient for effective optimization of fuzzy membership parameters in complex environments.

Table 2: Result for Complex Maps (Low Generation)

Metric	Baseline Fuzzy	GA-Fuzzy
Goal Reached	Never	Occasionally
Collisions	Frequent	Reduced but still frequent
Path Length	Long and inefficient	Slightly improved
Performance in Complex Map	Poor	Moderate

5.3 Results in Complex Map (High GA Generation)

With an increased number of GA generations, the performance of the GA–Fuzzy Controller improved significantly, as summarized in **Table 3**. The robot was able to reach the goal more consistently while exhibiting fewer collisions. The navigation trajectories were observed to be shorter and smoother compared to both the Baseline Fuzzy Controller and the low-generation GA–Fuzzy configuration.

This improvement indicates that the Genetic Algorithm was able to effectively tune the fuzzy membership parameters over successive generations. As a result, the robot demonstrated better decision-making when navigating through narrow and cluttered regions of the environment.

Table 3: Result for Complex Maps (High Generation)

Metric	Baseline Fuzzy	GA-Fuzzy
Goal Reached	Never	Frequently
Collisions	Frequent	Significantly reduced
Path Length	Long and inefficient	Shorter and smoother
Performance in Complex Map	Poor	Significantly improved

5.4 Overall Discussion

From the results obtained, the experimental results demonstrate that the Baseline Fuzzy Controller performs well in simple navigation environments with fewer obstacles and wider paths. In such conditions, the fixed fuzzy rules are sufficient to guide the robot safely to the goal.

However, in complex environments characterized by narrow passages and dense obstacle arrangements, the limitations of the Baseline Fuzzy Controller become evident. The fixed membership parameters restrict adaptability, leading to frequent collisions and navigation failure. The Hybrid GA–Fuzzy Controller addresses this limitation by automatically optimizing fuzzy membership parameters, resulting in improved obstacle avoidance, more stable navigation behaviour, and higher success rates in complex environments.

Overall, the results confirm that fuzzy logic alone is suitable for simple navigation tasks, while the integration of a Genetic Algorithm significantly enhances navigation performance in more challenging environments.

6.0 Conclusion and Future Works

This project successfully demonstrated the integration of fuzzy logic and genetic algorithms to optimize mobile robot navigation in both simple and complex environments. The Base Fuzzy Logic Controller provided reliable navigation in the Simple Map, allowing the robot to reach the goal without collisions. However, in complex environments with dense and irregular obstacles, the base controller alone was insufficient to ensure successful navigation. By introducing a Genetic Algorithm to optimize the fuzzy controller parameters, the Hybrid Fuzzy–GA system significantly improved performance, particularly when allowed sufficient evolutionary generations. High-generation optimization resulted in smoother trajectories, reduced total steps and time, and higher success rates in reaching the goal, illustrating the advantage of combining soft computing methods with evolutionary optimization.

The findings highlight the importance of adaptive control strategies for autonomous systems operating in uncertain and dynamic environments. While the Base Fuzzy Controller is intuitive and effective in straightforward scenarios, the GA-enhanced system demonstrates enhanced robustness and adaptability, reducing manual tuning efforts and improving overall navigation efficiency. Furthermore, the evolutionary approach allows the system to generalize across different obstacle configurations, providing a foundation for more intelligent autonomous behavior.

For future work, several improvements can be considered. The simulation can be extended to dynamic environments where obstacles are moving, requiring real-time adaptation of the fuzzy rules. Additional sensor modalities, such as range finders or vision-based input, could be incorporated to improve obstacle detection and path planning. Increasing the population size and diversity in the GA or experimenting with alternative optimization techniques such as particle swarm optimization or reinforcement learning could further enhance the controller's performance. Finally, implementing the system on a physical robot platform would provide practical insights and validate the effectiveness of the Hybrid Fuzzy–GA approach in real-world conditions.

7.0 Reference

Bingol, M. C. (2025). *A safe navigation algorithm for differential-drive mobile robots by using fuzzy logic reward function-based deep reinforcement learning*. *Electronics, 14*(8), 1593. https://doi.org/10.3390/electronics14081593

Hachani, S., & Nechadi, E. (2025). *Navigation system and obstacle avoidance for mobile robot using Type-2 fuzzy logic in uncertain environments*. *ITEGAM-JETIA, 11*(52), 68–75. https://doi.org/10.5935/jetia.v11i52.1534

Teng, Y., Feng, T., Song, C., Li, J., Yang, S. X., & Zhu, H. (2025). *Path planning of mobile robot based on dual-layer fuzzy control and improved genetic algorithm*. *Symmetry, 17*(4), 609. https://doi.org/10.3390/sym17040609

Saleh, M. S., Al Mashhadany, Y. I., Alshaibi, M., Ameen, F. M., & Algburi, S. (2025). *Optimal mobile robot navigation for obstacle avoidance based on ANFIS controller*. *Journal of Robotics and Control, 6*(1), 484–492.

Riman, C. F., & Abi-Char, P. E. (2024). *Novel fuzzy reinforcement algorithm for mobile robot navigation in automated storage*. *International Journal of Mechanical Engineering and Robotics Research, 13*(2), 284–295.

8.0 Appendix

Appendix A: AI Usage Declaration

A.1 AI Tools Used

Throughout the development of this project, several AI-based tools were used to assist in coding, debugging, simulation development, and report preparation. These tools were used as supportive aids while all final design decisions, parameter tuning, testing, and validation were performed by the group members.

The following AI tools were utilized:

AI Tool	Purpose of Use
Claude AI	Assisted in generating and structuring Python code for the fuzzy controller, GA training interface, and simulation framework.
GitHub Copilot	Assisted in refining code structure and suggesting improvements when tuning fuzzy membership values and rule implementations.
ChatGPT	Assisted in report writing, explanation of computational intelligence concepts, and improving technical documentation.
Antigravity (AI coding assistant)	Assisted in debugging code, resolving syntax/runtime errors, and ensuring smooth execution in PyCharm environment.

A.2 Examples of Key Prompts Used

Some examples of prompts given to the AI tools during the project are:

- "Generate a fuzzy logic controller for robot obstacle avoidance using triangular membership functions"
- "Help create a Genetic Algorithm to optimize fuzzy membership parameters"
- "Why is centroid defuzzification suitable for navigation control?"
- "Improve this Python code to make robot movement smoother"
- "Explain how to compare fuzzy and hybrid fuzzy-GA performance"
- "Help structure the report section for soft computing modeling"

These prompts were used to obtain initial ideas, code structure, and conceptual clarification.

A.3 How AI Outputs Were Used, Modified, and Verified

The outputs produced by AI tools were not used directly without modification. The group members:

- Modified the suggested code to fit the project architecture and simulation environment.
  Adjusted fuzzy membership values, rule bases, and GA parameters through multiple experiments.
- Verified the correctness of the algorithms by running simulations on both Simple and Complex maps.
- Debugged and refined the code manually to ensure accurate robot behaviour.
- Validated performance using evaluation metrics such as success rate, path efficiency, collision count, and time steps.

AI tools acted as assistive references, while the final implementation, testing, and validation were fully handled by the group.

A.4 Responsibility Statement

We confirm that we fully understand the algorithms, code structure, and results presented in this project. All final implementations, parameter selections, simulation results, and conclusions are the responsibility of the group members. AI tools were used only to assist learning, coding efficiency, and documentation, and not to replace our understanding or decision-making process.

Appendix B: Individual Contributions

This project was completed through a collaborative effort where each member contributed to different aspects of the system design, implementation, testing, and documentation.

Name	Contribution
Muhammad Hazimuddin bin Fairuz (2312479)	Designed and implemented the base fuzzy logic controller, including the development of membership functions, rule base formulation, and integration of sensor inputs for navigation. Assisted in simulation testing and validation on both maps.
Muhammad Zamarul Azim bin Zulkhibri (2312451)	Developed the Genetic Algorithm (GA) training framework for optimizing fuzzy membership parameters. Worked on fitness function design, GA parameter tuning, and performance evaluation between fuzzy and hybrid controllers.
Muhammad Aiman Alfaruq bin Noredan (2316963)	Developed the simulation environment, map design (simple and complex), GUI visualization, and comparison interface between Fuzzy and GA-Fuzzy controllers. Led the report writing, documentation, and result analysis.

All members were involved in:

- Testing and debugging the system in PyCharm,
- Analysing navigation performance,
- Verifying simulation results,
- Preparing the final report and presentation.

Each member fully understands the implementation and takes responsibility for the final outcome of the project.